

The Physics:

The Ordinary Differential Equation:

The ordinary differential equation (ODE) represented in this program is the single first-order ODE $dQ/dt = -Q/RC$. This is a model for a resistor-capacitor circuit: a charged capacitor discharges through a resistor, and as the current flows, the capacitor loses charge. In turn, the rate of charge loss depends on the amount of charge left in the capacitor.

The rate depends on the current charge because a charged capacitor has a voltage across it given by $V = Q/C$, where voltage is proportional to charge. The more charge stored, the more voltage pushing that charge out of the capacitor. That voltage drives a current out of the capacitor and through the resistor in accordance with Ohm's law, $V = IR$. By setting these two equations equal to one another through the voltage term, you get $IR = Q/C$. Rearranging this equation to isolate the current gives $I = Q/RC$. Since the capacitor is losing charge, the current corresponds to the decrease in charge over time, $I = -\Delta Q/\Delta t$ (or $-dQ/dt$), which gives the first-order differential equation used in this program: $dQ/dt = -Q/RC$. The Q/RC term is negative because charge is leaving the capacitor.

The physical meaning of this ODE is that the rate of discharge is proportional to how much charge is in the capacitor. Early on, when the capacitor is fully charged, the voltage is high, so current flows quickly and charge drains fast. As the charge drains, voltage drops and the current slows. The closer the charge gets to zero, the less urgency the capacitor has to discharge. This self-limiting behavior (the more you have, the faster you lose it, but the loss slows as you have less) produces an exponential decay curve, with the same underlying mechanism as radioactive decay.

The product RC is the time constant (τ) and indicates how fast the whole process occurs. A bigger resistor restricts current flow more, causing discharge to take longer, and a larger capacitor stores more charge for the same voltage, so it takes longer to drain. At $t = \tau$, the charge has dropped by roughly 63.2%, leaving about

36.8% of its starting value. This is a fixed, universal fact independent of the initial charge, resistance, and capacitance.

Integral:

Consider two point charges near each other, separated by a distance R . If they have the same sign, they repel, and if they have different signs, they attract. In either case, moving one charge relative to the other takes or releases energy, and that stored energy is called electric potential energy. While knowing the electric potential energy is great it can be more useful to know the work of the system. The work is the energy transferred when a force moves a charge a distance through an electric field.

At first, finding the work done might seem like a simple matter of multiplying force by distance. However, because of Coulomb's law, $F(r) = kq_1q_2/r^2$, the force is not constant. It decreases as the charges move further apart, following an inverse-square law (the same mathematical setup as gravity). Since the force changes continuously with separation, all of the tiny bits of work done as the charges move through each infinitesimally small increment of distance must be added up, and this is modeled mathematically with an integral. For this case the

integral is $W = \int_R^{\infty} (kq_1q_2/r^2) dr = (kq_1q_2)/R$.

The upper bound is infinity because potential energy needs a reference point, a place where the energy is defined as zero. The standard convention in electromagnetism is that two charges infinitely far apart do not interact at all, so their potential energy there is zero. The integral above computes how much work it would take to bring the charge from infinity down to separation R , which by definition is the potential energy stored at that separation. Even though the integral extends to infinity, the force weakens as $1/r^2$, so most of the work accumulates close to R . The contribution from far away shrinks quickly enough that the total stays finite.

The sign of the work depends on the signs of the two charges. If q_1 and q_2 have the same sign, W is positive, because energy must be put into the system to push the charges closer together against their natural repulsion. If they have opposite

signs, W is negative, because energy is released when the charges come together and energy is required to separate them.

The common thread:

Both problems come from the same family: a quantity is defined by a rate of change, and finding the total behavior over time or distance requires accumulating that rate continuously rather than using a constant rate.

The integration algorithms implemented:

Euler's Method: The Euler's method approximates the solution of an ODE ($dy/dt = f(t,y)$) by treating the curve as a series of small straight-line steps. At each point, it evaluates the slope given by the ODE at the current state, then walks forward one small time step (Δt) along that slope to estimate the next state: $y_{n+1} = y_n + \Delta t * f(t_n, y_n)$. This is the simplest ODE solver and as it goes on the method of only sampling the slope once per time step and assuming a constant slope within each step quickly breaks down if the actual slope changes too drastically inside a step or if the step size is too big for how fast the system evolves. This is why Euler's method is a first-order method: its error shrinks proportionally to the step size itself, so halving Δt roughly halves the error.

Runge Kutta 4th Order (RK4) Method: RK4 improves on Euler's method by sampling the slope at four different points within each step as opposed to once at the start, and combining those points into a single weighted average slope to represent that step. It evaluates the slope at the start of the interval (k_1), then at two estimates of the midpoint using increasingly refined guesses (k_2, k_3), and finally at an estimate of the endpoint (k_4): $k_1 = f(t_n, y_n)$, $k_2 = f(t_n + (\Delta t / 2), y_n + (\Delta t / 2) * k_1)$, $k_3 = f(t_n + (\Delta t / 2), y_n + (\Delta t / 2) * k_2)$, $k_4 = f(t_n + \Delta t, y_n + \Delta t * k_3)$, and $y_{n+1} = y_n + (\Delta t / 6) (k_1 + 2k_2 + 2k_3 + k_4)$. By sampling the slope's behavior across the whole interval rather than just the start, RK4 captures curvature in the solution that would be missed if Euler's method was used. This makes RK4 a forth-order method, its error shrinks proportionally to Δt^4 , so halving the step size reduces the error by a factor of roughly sixteen, which is

why RK4 achieves dramatically better accuracy than Euler's method even with the same number of steps.

Simple Riemann Sum: The simple Riemann sum approximates the area under a curve by dividing the intervals $[a,b]$ into n equal-width rectangles and using the function's value at one edge of each rectangle (in this project the left edge was used) as the height for that rectangle. The total area is the sum of all the

rectangles' areas: $\int_a^b f(x) dx \approx \Delta x \sum_{i=0}^{n-1} f(x_i)$. This is the most basic numerical

integration method, since it assumes the function stays constant across each rectangle, an assumption that gets worse the more the function experiences within each rectangle. For a strictly increasing or decreasing function, this produces a predictable, one-directional bias. For example our left-edge sum consistently overestimates a decreasing function and underestimates an increasing one.

Trapezoidal Rule: The trapezoidal rule approximates the area under a curve by dividing the interval $[a,b]$ into n equal-width stripes and treating each stripe as a trapezoid rather than a rectangle. It connects the function's values at each stripe endpoints with a straight line preserving the change in height from one side of the stripe to the other, instead of giving a flat top which could cut off a significant portion of the area. The area of each trapezoid is the average of its two parallel

heights times its width: $\int_a^b f(x) dx \approx (\Delta x)/2 [f(x_0) + 2f(x_1) + 2f(x_2) + \dots + 2f(x_{n-1}) +$

$f(x_n)]$. This is a more accurate way of getting the area under the curve than using the simple Riemann sum because it accounts for the function's slope within each interval as opposed to assuming the slope is constant.

Simpson's Rule: The Simpson's rule improves further than the trapezoid method by fitting a parabola through each stripe. Just like with the previous two methods the interval $[a,b]$ is divided into n equal-width stripes, but what sets Simpson's rule apart is that it takes three points from each stripe (the left edge, the right edge, and the mid-point) and applies a parabola rather than a straight line. The area under each parabolic segment works out to a weighted sum of the three points, with the middle point weighing four times more than the other two. The

equation is : $\int_a^b f(x) dx \approx (\Delta x)/3 [f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \dots + 4f(x_{n-1}) + f(x_n)]$. This method requires an even number of intervals to allow the points to come in complete groups of three, and it converges to the actual integral much faster than either the Riemann sum or the trapezoidal rule as the number of points increases.

Analytical Solutions:

RC Circuit:

The RC circuit's behavior is governed by the first-order ODE $dQ/dt = -Q/(RC)$, which can be solved exactly using separation of variables, a technique that works because the rate of change depends only on the current value of Q , not on time directly. Rearranging so that all Q terms sit on one side and all t terms on the other: $dQ/Q = -1/(RC) dt$. Then integrate both sides (the left with respect to Q and the right with respect to t): $\ln |Q| = -t/(RC) + c$. Exponentiating both sides and absorbing the constant of integration into a single coefficient (which the initial condition $Q(0) = Q_0$ fixes to be Q_0 itself) yields the closed-form solution: $Q(t) = Q_0 e^{-t/RC}$. This is an exact solution, not an approximation. For any t this formula gives the precise charge remaining on the capacitor, which is exactly why it serves as the ground truth for evaluating how accurate Euler, RK4, and SciPy's numerical solvers are. Physically, this equation says the charge decays exponentially, with the time constant $\tau = RC$ setting the pace of decay: at $t = \tau$, the charge has dropped to $1/e \approx 36.8\%$ of its starting value, regardless of what Q_0 actually is.

Electric Potential Energy:

The work needed to separate two point charges from distance R out to infinite separation is given by integrating Coulomb's force law over distance:

$W = \int_R^\infty \frac{kq_1q_2}{r^2} dr$. This integral can be solved exactly using the power rule, since

$1/r^2 = r^{-2}$ has a well-known antiderivative: $\int r^{-2} dr = -\frac{1}{r} + C$. Applying the

Fundamental Theorem of Calculus and evaluating at the bounds:

$W = kq_1q_2\left[-\frac{1}{r}\right]_R^\infty = kq_1q_2\left(0 - \left(-\frac{1}{R}\right)\right) = \frac{kq_1q_2}{R}$. The key step here is replacing r to ∞ as $-1/r$ to 0, even though the integral's upper bound is infinite, the integrand shrinks fast enough that the total accumulated area remains finite, which is what allows this improper integral to converge to a clean, closed-form answer rather than diverging. This exact result, $W = kq_1q_2/R$, is what the Riemann sum, trapezoidal rule, and Simpson's rule are all being checked against. With the understanding that the numerical methods must truncate the true infinite upper bound at some large but finite r_{max} , and are expected to converge toward this analytic value as r_{max} and the number of sample points both increase.

Results:

ODE:

$dQ/dt = -Q/(RC)$ was integrated with $R = 1000 \, \Omega$ and $C = 1.0 \times 10^{-3} \, F$ ($\tau = RC = 1.0 \, s$) and $Q_0 = 1.0 \, C$, using forward Euler, RK4, and SciPy's `solve_ivp`, and compared each with the analytic solution $Q(t) = Q_0e^{(-t/\tau)}$. For $0 \leq t \leq 10 \, s$ with $h = 0.05$, all three methods are visually indistinguishable from the exact curve (Fig. 1). I therefore quantify accuracy through the global error at t_{max} as a function of step size (Fig. 2).

The methods behave as their orders predict. Euler is first-order: its error at $t = 10 \, s$ is about $10^{-5} \, C$ at $h = 0.05$ and falls roughly in proportion to h . RK4 is fourth-order: its error falls from about $10^{-11} \, C$ at $h = 0.05$ to about $10^{-17} \, C$ at $h = 10^{-3}$, a slope near 4 on the log-log plot. `solve_ivp` is flat at about $10^{-6} \, C$ because it chooses its own adaptive steps, so its accuracy is set by its default tolerances and not by h . At these settings, RK4 is about six orders of magnitude more accurate than Euler at the same step size.

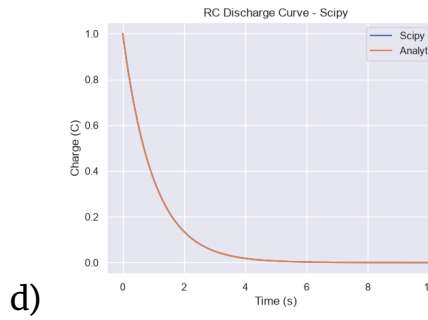
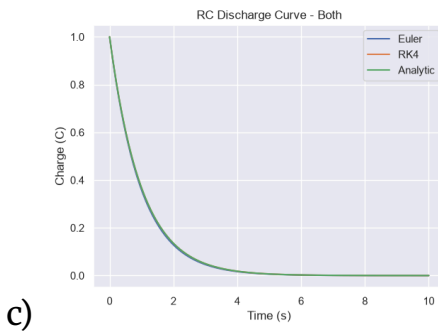
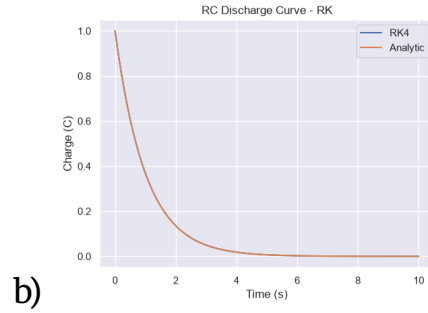
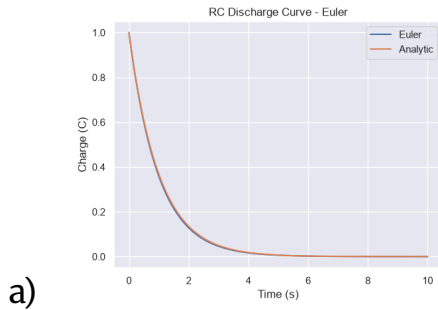


Figure 1: Charge vs. time for each method against the analytic solution; $R = 1000 \, \Omega$, $C = 1.0 \times 10^{-3} \, \text{F}$ ($\tau = RC = 1.0 \, \text{s}$), $Q_0 = 1.0 \, \text{C}$, over $0 \leq t \leq 10 \, \text{s}$ with $N = 200$ steps ($h = 0.05$).

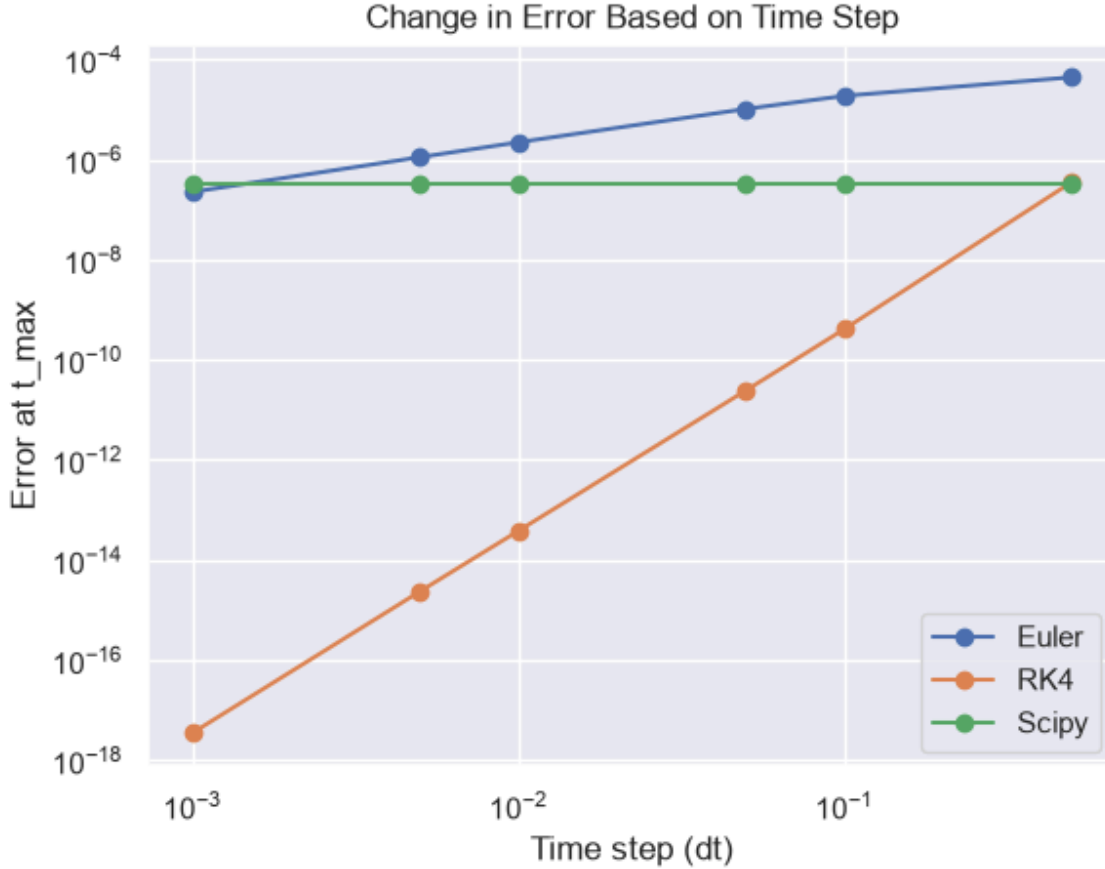


Figure 2: Change in error based on the time step, one curve per method; $R = 1000$ Ω , $C = 1.0 \times 10^{-3}$ F ($\tau = RC = 1.0$ s), $Q_0 = 1.0$ C, over $0 \leq t \leq 10$ s with $N = 200$ steps ($h = 0.05$).

$dQ/dt = -Q/(RC)$ was integrated with $R = 1000$ Ω and $C = 1.0 \times 10^{-3}$ F ($\tau = RC = 1.0$ s) and $Q_0 = 1.0$ C, using forward Euler, RK4, and SciPy's `solve_ivp`, and compared each with the analytic solution $Q(t) = Q_0 e^{-t/\tau}$. For $0 \leq t \leq 100$ s with $h = 0.02$, all three methods are visually indistinguishable from the exact curve (Fig. 3). I therefore quantify accuracy through the global error at $t_{\max}$ as a function of step size (Fig. 4).

Just like with the prior example the methods behave as their orders predict. Euler is first-order: its error at $t = 20$ s is about 10^{-5} C at $h = 0.02$ and falls roughly in

proportion to h . RK4 is fourth-order: its error falls from about 10^{-11} C at $h = 0.02$ to about 10^{-17} C at $h = 10^{-3}$, a slope near 4 on the log-log plot. `solve_ivp` is flat at about 10^{-6} C because it chooses its own adaptive steps, so its accuracy is set by its default tolerances and not by h . At these settings, RK4 is about six orders of magnitude more accurate than Euler at the same step size.

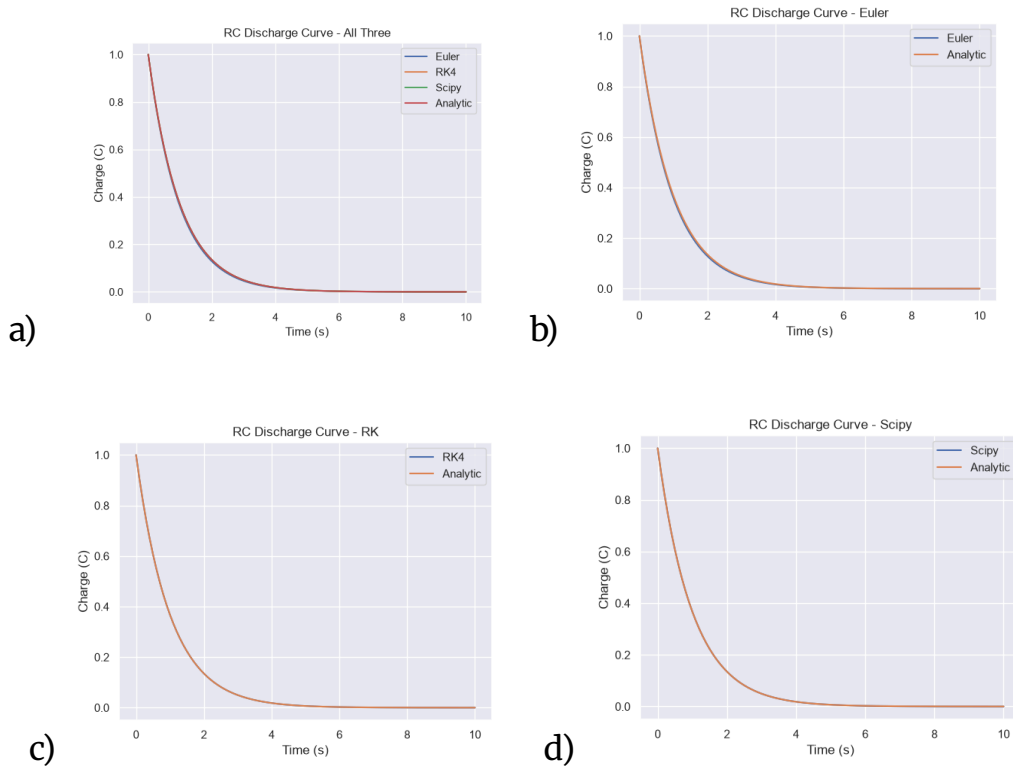


Figure 3: Charge vs. time for each method against the analytic solution; $R = 1000 \, \Omega$, $C = 1.0 \times 10^{-3} \, \text{F}$ ($\tau = RC = 1.0 \, \text{s}$), $Q_0 = 1.0 \, \text{C}$, over $0 \leq t \leq 100 \, \text{s}$ with $N = 5000$ steps ($h = 0.02$).

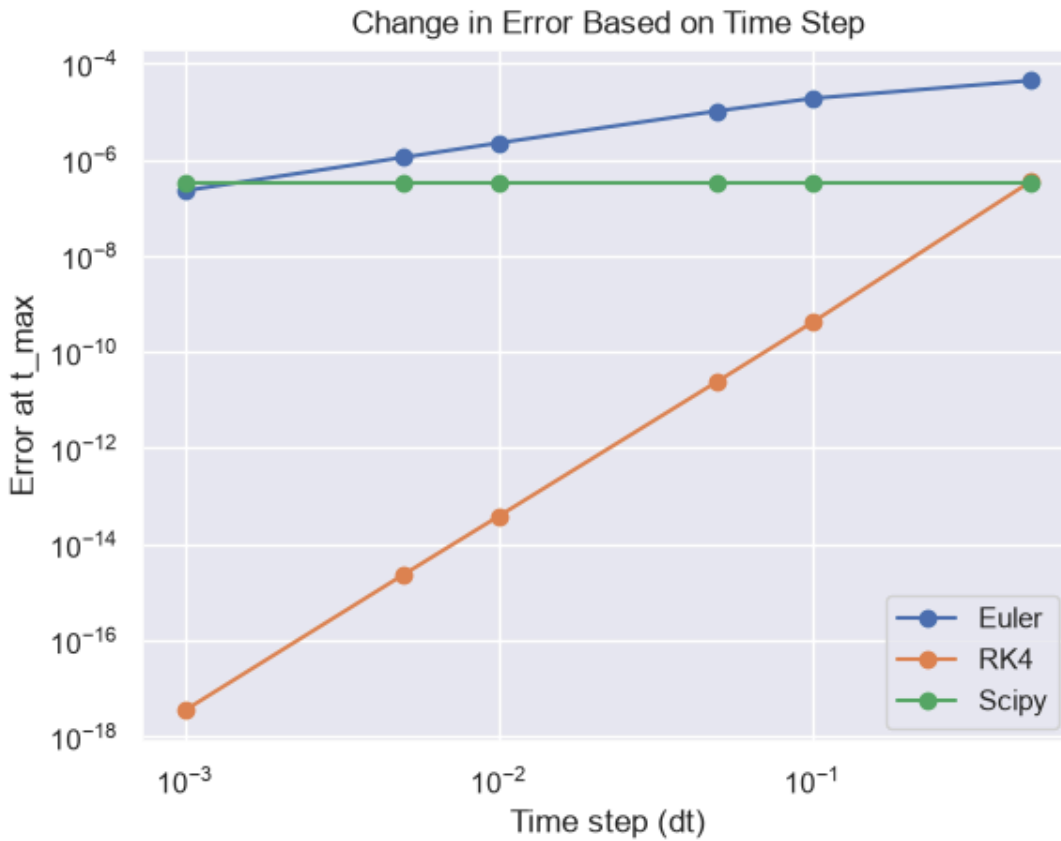


Figure 4: Change in error based on the time step, one curve per method; $R = 1000 \, \Omega$, $C = 1.0 \times 10^{-3} \, \text{F}$ ($\tau = RC = 1.0 \, \text{s}$), $Q_0 = 1.0 \, \text{C}$, over $0 \leq t \leq 100 \, \text{s}$ with $N = 5000$ steps ($h = 0.02$).

Integral:

$W = \int$ was computed from R to r_{max} of $kq_1q_2/r^2 \, dr$, with $k = 8.99 \times 10^9 \, \text{Nm}^2/\text{C}^2$ and $q_1 = q_2 = 1.0 \times 10^{-6} \, \text{C}$. The exact value is $W = kq_1q_2(1/R - 1/r_{\text{max}})$. I used a left Riemann sum, the trapezoidal rule, and Simpson's rule on a uniform grid, and checked each against SciPy's `trapezoid` and `simpson`. My implementations match SciPy to all printed digits.

Case 1 ($R = 0.01 \, \text{m}$, $r_{\text{max}} = 10 \, \text{m}$, $n = 5000$, $h \approx 2.0 \times 10^{-3} \, \text{m}$; exact $W \approx 0.8979 \, \text{J}$). The methods rank by order: the left Riemann sum (first-order) is off by about 10.7%, the trapezoidal rule (second-order) by about 0.66%, and Simpson's rule (fourth-order) by about 0.02%. The measured trapezoidal order of 1.43 is below

the theoretical 2 because the integrand is very steep near $r = R$ (Figure 5 and Table 1).

Case 2 ($R = 0.001$ m, $r_{\text{max}} = 20$ m, $n = 10000$; exact $W \approx 8.987$ J). Every method does much worse, with errors of 147% (Riemann), 47% (trapezoid), and 19% (Simpson). The reason is that a uniform step of 2×10^{-3} m is larger than R itself, so the first few intervals, where the integrand changes by orders of magnitude, are badly under-resolved. The measured trapezoidal order drops to 1.05. Uniform grids are a poor choice for this integrand. A logarithmically spaced grid, a substitution such as $u = \ln r$, or adaptive quadrature would fix it (Figure 6 and Table 1).

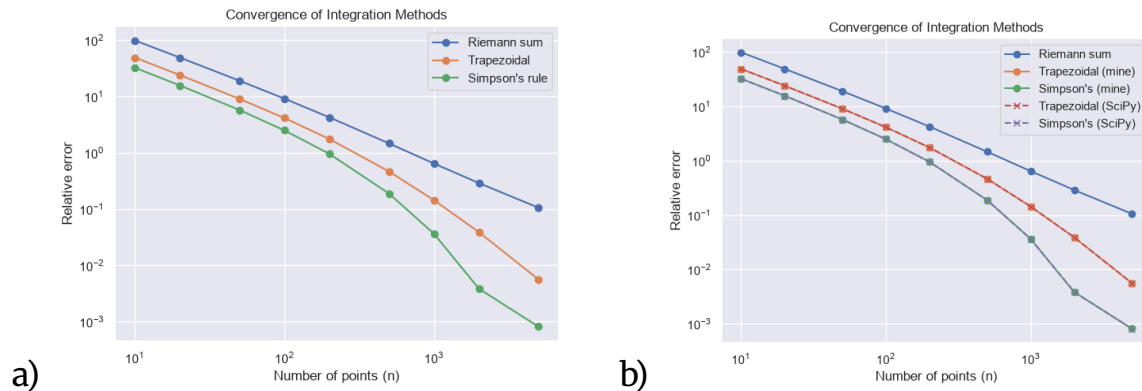


Figure 5. Case 1 a) The convergence of the Riemann sum, Trapezoidal sum, and Simpson's rule integration methods for $q_1 = 1.0\text{e-}6$ coulombs, $q_2 = 1.0\text{e-}6$ coulombs, $R = 0.01$ meters, $r_{\text{max}} = 10$ meters, n stripes = 5000 . b) The convergence of the Riemann sum, Trapezoidal sum, and Simpson's rule integration methods with the SciPy results graphed on top for $q_1 = 1.0\text{e-}6$ coulombs, $q_2 = 1.0\text{e-}6$ coulombs, $R = 0.01$ meters, $r_{\text{max}} = 10$ meters, n stripes = 5000 .

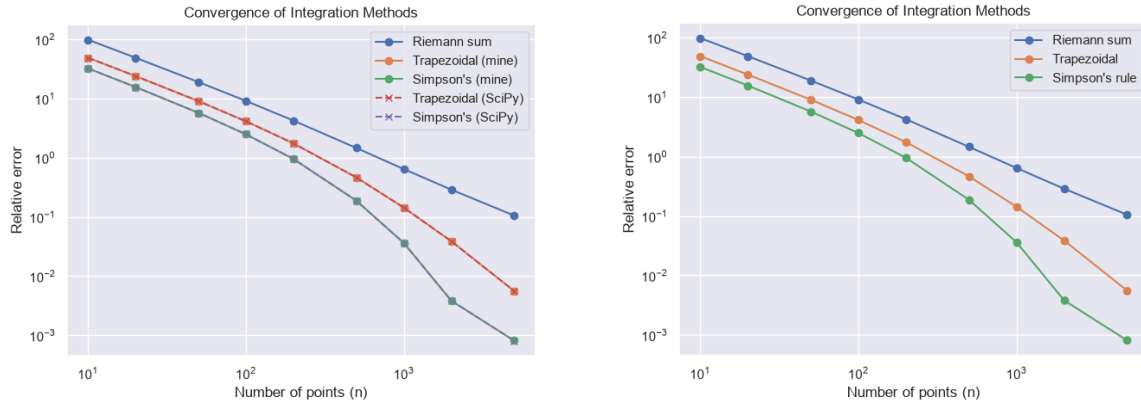


Figure 6. Case 2 a) The convergence of the Riemann sum, Trapezoidal sum, and Simpson's rule integration methods for $q_1 = 1.0\text{e-}6$ coulombs, $q_2 = 1.0\text{e-}6$ coulombs, $R = 0.01$ meters, $r_{\text{max}} = 20$ meters, n stripes = 10000 . b) The convergence of the Riemann sum, Trapezoidal sum, and Simpson's rule integration methods with the SciPy results graphed on top for $q_1 = 1.0\text{e-}6$ coulombs, $q_2 = 1.0\text{e-}6$ coulombs, $R = 0.01$ meters, $r_{\text{max}} = 20$ meters, n stripes = 10000 .

Table 1: The estimated value, real value and relative error for methods.

Case	Method	Estimated Value (J)	Analytic Value (J)	Relative Error
1	Riemann sum	9.935695e-01	8.987500e-01	1.055e-01
1	Trapezoidal sum	9.037845e-01	8.987500e-01	5.602e-03
1	Simpsons rule	8.980198e-01	8.987500e-01	8.125e-04
1	SciPy Trapezoidal sum	9.037845e-01	8.987500e-01	5.602e-03

1	SciPy Simpsons rule	8.980198e-01	8.987500e-01	8.125e-04
2	Riemann sum	2.217454e+01	8.987500e+00	1.467e+00
2	Trapezoidal sum	1.318749e+01	8.987500e+00	4.673e-01
2	Simpsons rule	1.069531e+01	8.987500e+00	1.900e-01
2	SciPy Trapezoidal sum	1.318749e+01	8.987500e+00	4.673e-01
2	SciPy Simpsons rule	1.069531e+01	8.987500e+00	1.900e-01

Validation:

Each implementation was checked against its analytic solution and against at least two properties that do not depend on the numerical method. Parameters are those used above unless noted.

RC discharge:

Agreement with the analytic solution. The global error at $t_{\max}$ shrinks with the expected power of h (Fig. 2). Between successive step sizes, the log–log slope of Euler approaches 1.00 (0.95–1.00 for $h \leq 0.05$), and the slope of RK4 is 4.0 (3.95–4.06). The closed-form error predictions in the error section also match the measured errors to about 10%.

Property 1: the time constant. For any exponential decay, $Q(\tau)/Q_0 = 1/e = 0.367879$, independent of Q_0 , R and C . I ran $(R, C) = (1000 \, \Omega, 1 \, \text{mF})$, $(100 \, \Omega, 10 \, \text{mF})$ and $(10 \, \Omega, 100 \, \text{mF})$, which all have $\tau = 1 \, \text{s}$. The curves overlap, so the solution depends only on t/τ . At $h = 0.01$, $Q(\tau)/Q_0$ is 0.366032 for Euler and 0.367879441 for

RK4, against an exact 0.367879441. Euler's deficit of 5×10^{-4} is what a first-order method should give at $h = 0.01$ (Error Section).

Property 2: positivity and stability. Physically, Q must decay monotonically and never change sign. For the linear ODE, one Euler step multiplies Q by $(1 - h/\tau)$. Euler therefore preserves this behavior only for $h < \tau$. At $h = \tau$ it drops to zero in one step, for $\tau < h < 2\tau$ it oscillates in sign while decaying, and for $h > 2\tau$ it diverges. With $h = 2.5$ s over 10 s (four steps), Euler gives $Q = (-1.5)^4 \approx 5.06$ C, against an exact 4.5×10^{-5} C. I verified that the code reproduces these regimes. This is the limiting case where the step size stops being small compared with τ , and it is treated further in the code performance section.

Property 3: charge and energy conservation. All charge leaving the capacitor must pass through the resistor, $\int_0^T I \, dt = Q_0 - Q(T)$, and the energy dissipated must equal the stored energy, $\int_0^T I^2 R \, dt = Q_0^2/2C = 500$ J. I computed both integrals from the numerical $Q(t)$ with $I = Q/RC$ using the trapezoid rule on the sampled data. For Euler, the relative discrepancy is 2.5×10^{-2} , 5.0×10^{-3} and 5.0×10^{-4} for $h = 0.05$, 0.01 and 0.001 , first-order in h and equal to $h/2\tau$. For RK4, the discrepancy is $\leq 8.3 \times 10^{-4}$ and falls as h^2 . It is set by the trapezoid rule used in the check, not by the ODE solver (Table 2).

Table 2: Relative discrepancy in the conservation checks ($T = 10$ s, $\tau = 1$ s), computed from the numerical $Q(t)$ with the trapezoid rule.

h (s)	Method	Charge: $\int I \, dt$ vs $Q_0 - Q(T)$	Energy: $\int I^2 R \, dt$ vs $Q_0^2/2C$
0.05	Euler	2.50×10^{-2}	2.44×10^{-2}
0.05	RK4	2.08×10^{-4}	8.33×10^{-4}
0.01	Euler	5.00×10^{-3}	4.97×10^{-3}
0.01	RK4	8.33×10^{-6}	3.33×10^{-5}
0.001	Euler	5.00×10^{-4}	5.00×10^{-4}
0.001	RK4	8.33×10^{-8}	3.33×10^{-7}

Work integral:

Agreement with SciPy. My left-Riemann, trapezoid and Simpson implementations reproduce `scipy.integrate.trapezoid` and `simpson` to all printed digits (Table 1).

Property 1: exactness on polynomials. The trapezoid rule integrates linear functions exactly, and Simpson's rule integrates cubics exactly. On $[0, 2]$ with 10 panels, my trapezoid gives $\int (3x+1)dx = 8$ and my Simpson gives $\int x^3 dx = 4$, both to round-off.

Property 2: scaling and sign. W must scale as $q_1 q_2$ and as $1/R$, and reversing the sign of one charge must reverse the sign of W without changing its magnitude. Doubling both charges multiplies W by 4, and $W(-q_1, q_2) = -W(q_1, q_2)$. Both hold to round-off. These two checks mainly test that the parameters are wired in correctly, because $kq_1 q_2$ factors out of every quadrature rule.

Property 3: the $r_{\text{max}} \rightarrow \infty$ limit. The exact truncated result is $W = kq_1 q_2 (1/R - 1/r_{\text{max}})$. It approaches the infinite-separation value $kq_1 q_2 / R$ with a fractional deficit of exactly R/r_{max} . That deficit is 10^{-3} for Case 1 and 5×10^{-5} for Case 2, and the code reproduces both.

Code Performance:

A case in which my code performs noticeably worse is **Integral, Case 2** ($R = 0.001 \text{ m}$, $r_{\text{max}} = 20 \text{ m}$, $n = 10^4$). This is the clearest failure. All three rules are off by 19–147%, against 0.02–10.7% in Case 1. This is not a coding error, because my implementations still match SciPy to all digits. It is a limitation of the method on a uniform grid.

The step is $h = (r_{\text{max}} - R)/n \approx 2 \times 10^{-3} \text{ m}$, which is twice R . The integrand $kq_1 q_2 / r^2$ falls by a factor of 9 between the first and second sample points. The first panel $[0.001, 0.003]$ therefore dominates the answer, and none of the rules can represent it (Table 3).

The remaining 9,999 panels contribute comparatively little error. The convergence orders degrade for the same reason. Between $n = 10$ and $n = 5000$, the trapezoid order is about 1.05–1.1 in Case 2, against 1.43 in Case 1, and Simpson's rule is barely better than first-order. The asymptotic error formulas in Section 7 require $h \ll R$, and that fails here ($h/R = 2$).

Fix. The remedy is to resolve the region near R , not to add points everywhere. With the same $n = 10^4$, a geometrically spaced grid (`np.geomspace(R, r_max, n+1)`) gives a trapezoid relative error of $\approx 5 \times 10^{-7}$.

Table 3: First panel contribution, rule contribution, and panel error for Case 2 of the integral problem.

	Exact contribution of first panel	Rule's contribution	Panel error as share of the rule's total error
Left Riemann	5.99 J	17.98 J	$\approx 91\%$ (12.0 of 13.2 J)
Trapezoid	5.99 J	9.99 J	$\approx 95\%$ (4.0 of 4.2 J)

Another area where my code is noticeably worse is in the **ODE, when there is a large step relative to τ** . The Euler method is noticeably worse than `solve_ivp` when h is not small compared with τ . At $h = 2.5$, Euler diverges while `solve_ivp` (which chooses its own steps) does not. RK4 stays stable but is still inaccurate: its per-step factor is 0.648, so $Q(10) = 0.177$ against an exact 4.5×10^{-5} . RK4's real-axis stability limit is $h \approx 2.79$. The controlling parameter is h/τ , not h itself.

Errors: Truncation, Observed, and Sources:

Let $z = h/\tau$. The local truncation error is the error made in one step from exact data, and the global error is what accumulates over $N = T/h$ steps.

Euler: Expanding, $Q(t+h) = Q + hQ' + (h^2/2)Q'' + \dots$. Euler keeps only the first two terms, so the local error is $(h^2/2)Q'' = O(h^2)$. For this ODE, $Q'' = Q/\tau^2$, so the local error is $(z^2/2)Q$. The errors accumulate over $N \propto 1/h$ steps, so the global error is $O(h)$. For this ODE it can be found exactly. Euler gives $Q_N = Q_0(1 - z)^N$, and $\ln(1 - z) = -z - z^2/2 - \dots$, so $Q_N \approx Q_0 e^{-(T/\tau)} [1 - hT/(2\tau^2)]$, which means Euler underestimates by $\approx Q_0 e^{-(T/\tau)} hT/(2\tau^2)$ (Table 4).

RK4: For this linear ODE one RK4 step multiplies Q by the fourth-order Taylor polynomial $R(z) = 1 - z + z^2/2 - z^3/6 + z^4/24$. The exact factor is $e^{-z} = R(z) - z^5/120 + \dots$. The local error is therefore $\approx (z^5/120)Q = O(h^5)$, and after N steps the global error is $\approx Q_0 e^{-(T/\tau)} (T/\tau) z^4/120 = O(h^4)$. RK4 overshoots slightly (Table 4).

Quadrature rules. For a panel width h , the standard results are left Riemann's local error $(h^2/2)f'(\xi)$ and global $(h/2)[f(b) - f(a)] + O(h^2)$, trapezoid's local $-(h^3/12)f''(\xi)$ and global $(h^2/12)[f'(b) - f'(a)] + O(h^4)$ for the overestimate $S - I$, and Simpson's local $-(h^5/90)f'''(\xi)$ per pair of panels and global $(h^4/180)[f'''(b) - f'''(a)]$ for $S - I$. For $f = kq_1q_2/r^2$, the derivatives at $r = R$ dominate, giving the error estimates below (Table 4).

Table 4: The local and global error of each method from both problems accompanied by the predicted issue.

Method	Local error	Global error	Predicted for this problem
Euler	$O(h^2)$	$O(h)$	$Q_0 e^{-(T/\tau)} hT/(2\tau^2)$
RK4	$O(h^5)$	$O(h^4)$	$Q_0 e^{-(T/\tau)} (T/\tau) (h/\tau)^4/120$
Left Riemann	$O(h^2)$ per panel	$O(h)$	$(h/2)kq_1q_2/R^2$ (+ trapezoid term)
Trapezoid	$O(h^3)$ per panel	$O(h^2)$	$h^2kq_1q_2/(6R^3)$
Simpson	$O(h^5)$ per pair	$O(h^4)$	$2h^4kq_1q_2/(15R^5)$

Some errors were found through observation. For the ODE problem, at $t = 10$ s and $h = 0.05$, the predicted Euler error is 1.13×10^{-5} C against 1.03×10^{-5} C

measured. The predicted RK4 error is 2.4×10^{-11} C against 2.5×10^{-11} C measured. The slopes in Fig. 2 are 1.0 and 4.0 as derived. The Euler slope is only ≈ 0.5 at the two largest h because ht/τ is not small there and the first-order expansion no longer holds.

At $h = 10^{-3}$ the RK4 point (measured $\approx 4.1 \times 10^{-18}$ C, predicted 3.8×10^{-18} C) is beginning to show round-off, so I did not go to smaller h .

In the integral problem the quadrature of case 1 was in issue. The Simpson prediction is within about 10%, which is reasonable because $h/R = 0.2$ is not asymptotically small (Table 5).

The overall measured orders from $n = 10$ to $n = 5000$ are 1.1, 1.43 and 1.94. Your report currently treats the shortfall as a defect of the method. The local slopes show it is a pre-asymptotic effect. Between successive n , the trapezoid slope climbs from 1.03 at small n to 1.96 at $n = 2000$ –5000, and Simpson's slope climbs from 1.04 to 3.55. Most of the n range has $h > R$. Only at the largest n does h fall below R and the theoretical orders emerge. I suggest adding a table of these local slopes to demonstrate the correct scaling.

Table 5: Predicted and measured error for case 1 of the integral problem.

Rule	Predicted S - I	Measured S - I
Left Riemann	0.0898 J (0.0958 J with the trapezoid term added)	0.0957 J
Trapezoid	5.98×10^{-3} J	5.93×10^{-3} J
Simpson	1.91×10^{-4} J	1.69×10^{-4} J

The sources of these errors vary based on the type of error. For the truncation error it is set by the method and by h . For the quadratures, the size of h relative to the scale on which the integrand varies (here R) is what matters, not h itself.

In a similar fashion the domain truncation comes from replacing the infinity bound of the integral with r_{max} , this removes kq_1q_2/r_{max} , a systematic fractional deficit of R/r_{max} (10^{-3} for Case 1). It is identical for all methods and sets a floor when comparing against the infinite-limit value. There was also error caused by round-off, this is relatively negligible for the quadratures but for RK4 it sets a floor once the truncation falls below roughly 10^{-14} .

Improvements can be made by using a smaller step size or a higher-order method for the ODE and using a non-linear grid (geometric or substitution) or adding an analytic tail kq_1q_2/r_{max} to remove the domain truncation error from the integral.